Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

For this project, I'm going to be using lists, loops, functions, and conditionals to calculate the percentage of observed Bitcoin blocks. I want to improve my application/understanding of python, and what i've been learning the past semester in cisc106; i want to master using drafter and usig it's library, input and state fields, loops, and if statements.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest part of the project will definitely be using loops and if statements to determine each mining pool's percentage of blocks minded. We've already used loops, lists, and conditionals in the past. However, I'm not too familiar with drafter so the hardest part of the project will be having to connect the pages, manage State, and make sure the results update correctly based on the inputs.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Building the website itself with drafter and coding the calculations to user inputs will take the most time. A MiningPool dataclass and a saved dataset containing minind pool names and block counts is what i plan to start with, then I'll create functions to calculate the percentages, rank the mining pools, and compared their combined share against a threshold. Once they're all working, I'll build the starting pages and routes in Drafter.

D. What distractions might interfere with your work, and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

100% other assignments will distract me and interfer. I'll tyr to adovid them by working on the project in smaller parts separately and setting aside time to code. I'm also gonna utilize chatgpt and past assignments to help when I get stuck on understanding something or an error. I don't have any questions.

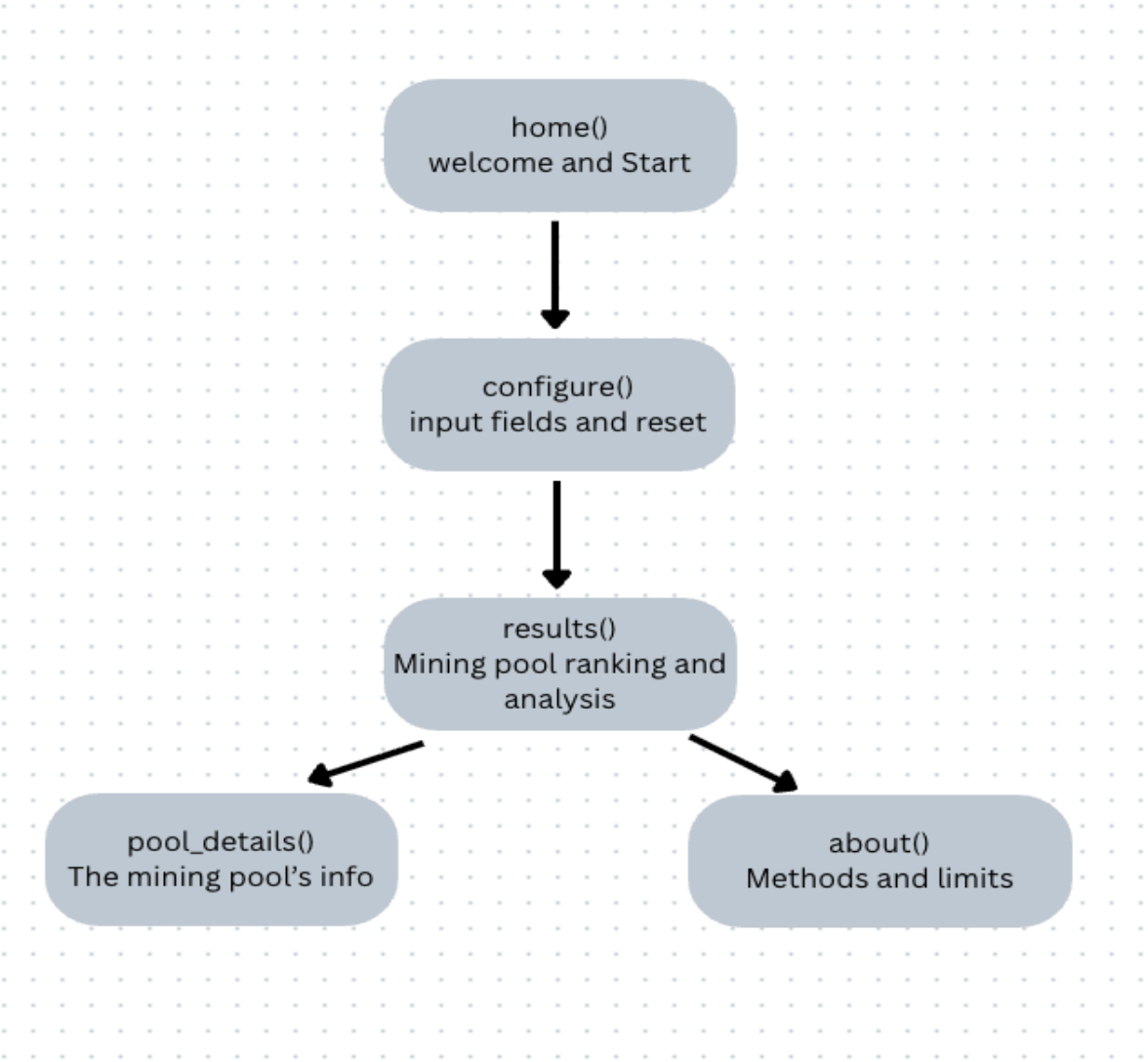
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram to show that you have some kind of idea.

We recommend drawing each page as a rectangle and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



home() : website title, project description, and Start button

configure(): Top pools TextBox, threshold TextBox, mining pool SelectBox, Analyze and Reset buttons

results(): Ranked mining pool table, calculated percentages, combined share, concentration explanation, View Pool and About buttons

pool_details(): Selected pool name, blocks mined, percentage share, observation period, Back button

about(): Explanation of the calculations, what mining concentration means, limitations of the data, Home button

Value types:
pools: list[MiningPool]
top_count: int
threshold: float
selected_pool: str
combined_share: float

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The website will analyze Bitcoin mining pool concentration using a saved dataset of observed blocks. Users will be able to select how many of the largest mining pools they want to examine and compare their combined share against a percentage threshold to better understand mining concentration.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	pools (list[MiningPool]): A list containing the mining pool records from my saved dataset. Each pool's calculated share will be updated when the analysis runs. top_count (int): The number of largest mining pools the user wants to analyze. It will change based on the user's input. threshold (float): The percentage threshold the user wants to compare against. This will be updated when the user enters a different threshold. selected_pool (str): The name of the mining pool selected by the user. It will change when the user chooses another pool to inspect. combined_share (float): The combined percentage of blocks mined by the selected number of largest pools. It will be recalculated each time the user runs the analysis.

Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	1. **name (str):** The name of the mining pool. 2. **blocks_mined (int):** The number of blocks produced by that mining pool during the observation period. 3. **period (str):** The time period when the mining data was recorded. 4. **share (float):** The calculated percentage of observed blocks produced by the pool.
Routes (minimum 7) The names of your route functions	home(): Displays the homepage with a description of the website. configure(): Displays the page where users enter their analysis settings. analyze(): Processes the user inputs, calculates mining concentration, and updates State. results(): Displays the ranked mining pools and concentration results. pool_details(): Displays information about an individual mining pool. about(): Explains how the calculations work and what the results mean. reset_settings(): Resets the user's inputs to their default values and returns to the configuration page.
Pages (minimum 5) A brief description of the pages in your site	1. **Home Page:** Introduces the Bitcoin Mining Decentralization Analyzer and explains its purpose. It will have a button to begin the analysis. 2. **Configuration Page:** Allows users to enter the number of largest pools to examine, set a percentage threshold, and select a mining pool.

	3. **Results Page:** Displays the ranked mining pools, individual percentages, combined percentage, and whether the result exceeds the selected threshold. 4. **Pool Details Page:** Displays additional information about a selected pool, including its name, number of blocks mined, and percentage of the total. 5. **About Page:** Explains the project's calculations, data source, and limitations when measuring Bitcoin mining decentralization.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	TextBox: Allows users to enter the number of largest mining pools they want to analyze, such as the top 3 or top 5. TextBox: Allows users to enter a percentage threshold, such as 50%, to compare against the combined mining share. SelectBox: Allows users to select an individual mining pool from the dataset to view more information about that pool.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	The if statements will be used to validate the number of pools. Whether the user entered valid number of mining pools or not; if the number is les than 1 or greater thant he number of pools available. There will be an error. It will also validate the threshold checking if the user's percentage is between 0 and 100. The last thing an if statement will be used for is for comparing the combined percentage of the largest pools.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	A for loop will be used to iterate through the list of MiningPool dataclass instances stored in State. The loop will help calculate the total number of observed blocks and each mining pool's percentage of the total. I'll also use the mining pool data to rank pools from largest to smallest and calculate the combined percentage of the top pools selected by the user. This is important because the website needs to

	process multiple mining pools instead of calculating each one individually.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I continued the last conversation I had with CHATGPT.

I'm going to submit the whole conversation since the beginning.

CS1 Final Project 0_ Pick a Problem.pdfPDF

Final Project: 0 - Selection

- Due Friday by 4pm
- Points 20
- Submitting a file upload
- File Types doc, docx, and pdf
- Available Oct 16 at 12am - Nov 6 at 11:59pm

This assignment is meant to be done during lecture. Do not begin this assignment prior to the start of class.

Please make sure to review the Final Project Requirements first.

The assignment Google Doc for this activity is:

https://docs.google.com/document/d/1duohCtzsD0c4yVUMJoHovfNH7-bu52aaXcj2qzzs0Uc/copyLinks to an external site.Links to an external site.

Everyone should submit their own worksheet when complete. Do NOT share worksheets.

You are allowed to use in this activity. If you do, you must submit the entire conversation with the LLM in the worksheet above.

If You Missed This Activity...

If you missed this activity, attend office hours to makeup the points.

Rubric

Final Project 2026 - part 0

for some project ideas and interests I have is blockchain and bitcoin. generate me research ideas i could possible pursue in those fields for this final project for my cisc106 class.

For your CISC106 Final Project, I think Bitcoin, blockchain, and decentralization are excellent subjects to explore, especially if you want to build something that connects computer science with economics and finance.

After reviewing your uploaded worksheet, there is one important distinction: you're not being asked to conduct a full research study. You're being asked to build a functional Python web application using Drafter that solves a specific problem.

Your rubric emphasizes:

- A clearly defined problem and practical purpose.
- Specific inputs, calculations, and outputs using Python concepts.
- Data abstraction, ideally using dataclasses with 3–4 meaningful attributes.
- A realistic scope that you can implement, test, and debug.

Your worksheet also explicitly asks you to research existing solutions, identify overly

complicated features, and refine your original idea.

CS1 Final Project 0_ Pick a Problem.pdf

CS1 Final Project 0_ Pick a Problem.pdf

I'd focus on applications that explain or simulate how blockchain systems work, rather than trying to develop a real cryptocurrency, network, or trading platform.

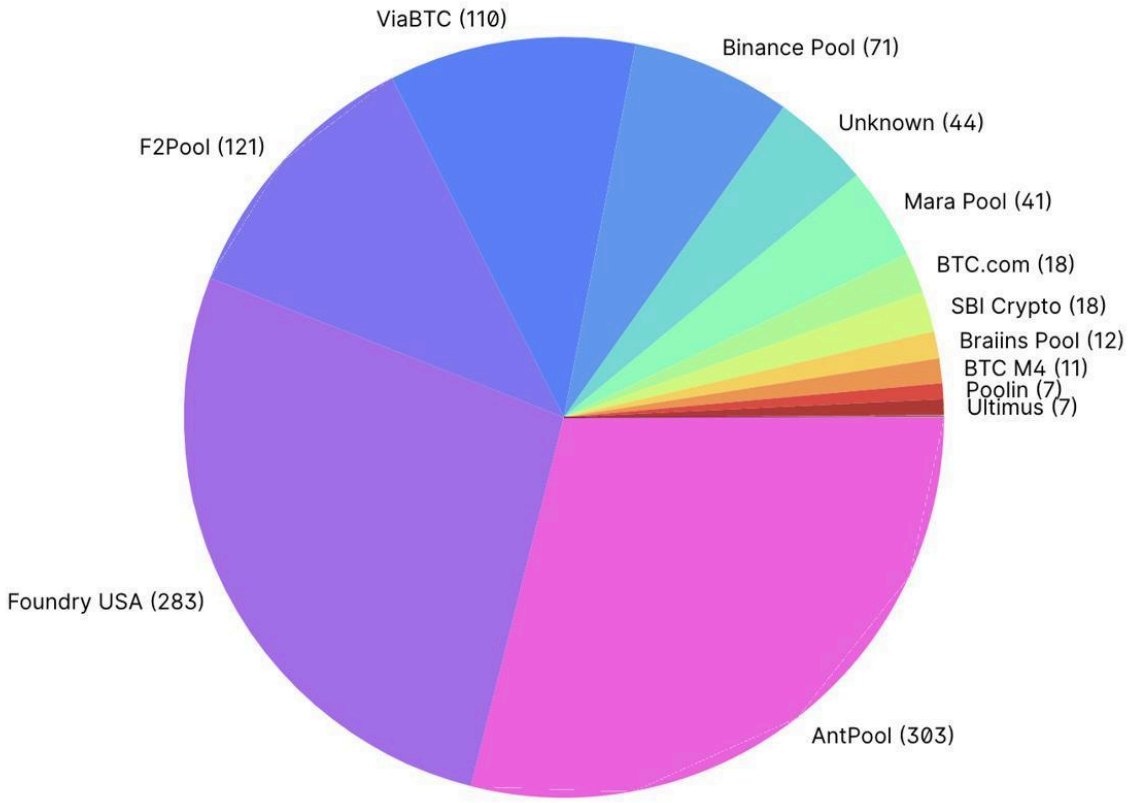
7 Bitcoin and blockchain project ideas

I've ranked these based on how interesting they could be, how well they fit the rubric, and how difficult they would be to build.

Project	Research question and web application	Difficulty
1. Bitcoin Mining Decentralization Analyzer My top pick	How concentrated is Bitcoin mining among the largest pools? Build an app that compares mining pools, calculates percentage shares, and identifies how many of the largest pools account for over 50% of observed blocks.	Medium
2. Bitcoin Mining Profitability Simulator	How do electricity prices and mining hardware affect profitability? Let users enter hardware specifications and assumptions, then calculate estimated operating costs, earnings, and break-even points.	Easy–Medium
3. Blockchain Security and Tampering Simulator	Why is changing information in a blockchain detectable? Let users create a small chain of blocks, alter stored information, and see how hash verification detects changes.	Medium
4. Bitcoin Transaction Fee Calculator	How does transaction size affect Bitcoin network fees? Let users enter a transaction's virtual size and compare estimated costs at different fee rates.	Easy
5. Bitcoin Halving and Supply Simulator	How does Bitcoin's fixed issuance schedule affect the supply of new coins? Let users enter a block height and calculate the block subsidy, upcoming halving, and estimated issuance.	Easy

6. Blockchain Consensus Security Simulator	How many faulty or malicious validators can a blockchain tolerate? Simulate a simplified Byzantine fault-tolerant consensus system and evaluate validator voting power against its security threshold.	Medium
7. Bitcoin Node Decentralization Dashboard	How geographically distributed is Bitcoin's publicly reachable node network? Analyze node counts across countries, calculate geographic shares, and compare concentrations.	Easy–Medium

Three projects I would seriously consider



1. Bitcoin Mining Decentralization Analyzer

This would be my first choice for you. It's connected to questions about decentralization, mining incentives, and Bitcoin's security. You could investigate how

much mining is concentrated among a small number of pools, using actual historical data from

mempool.space

mempool.space

+1



2. Bitcoin Mining Profitability Simulator

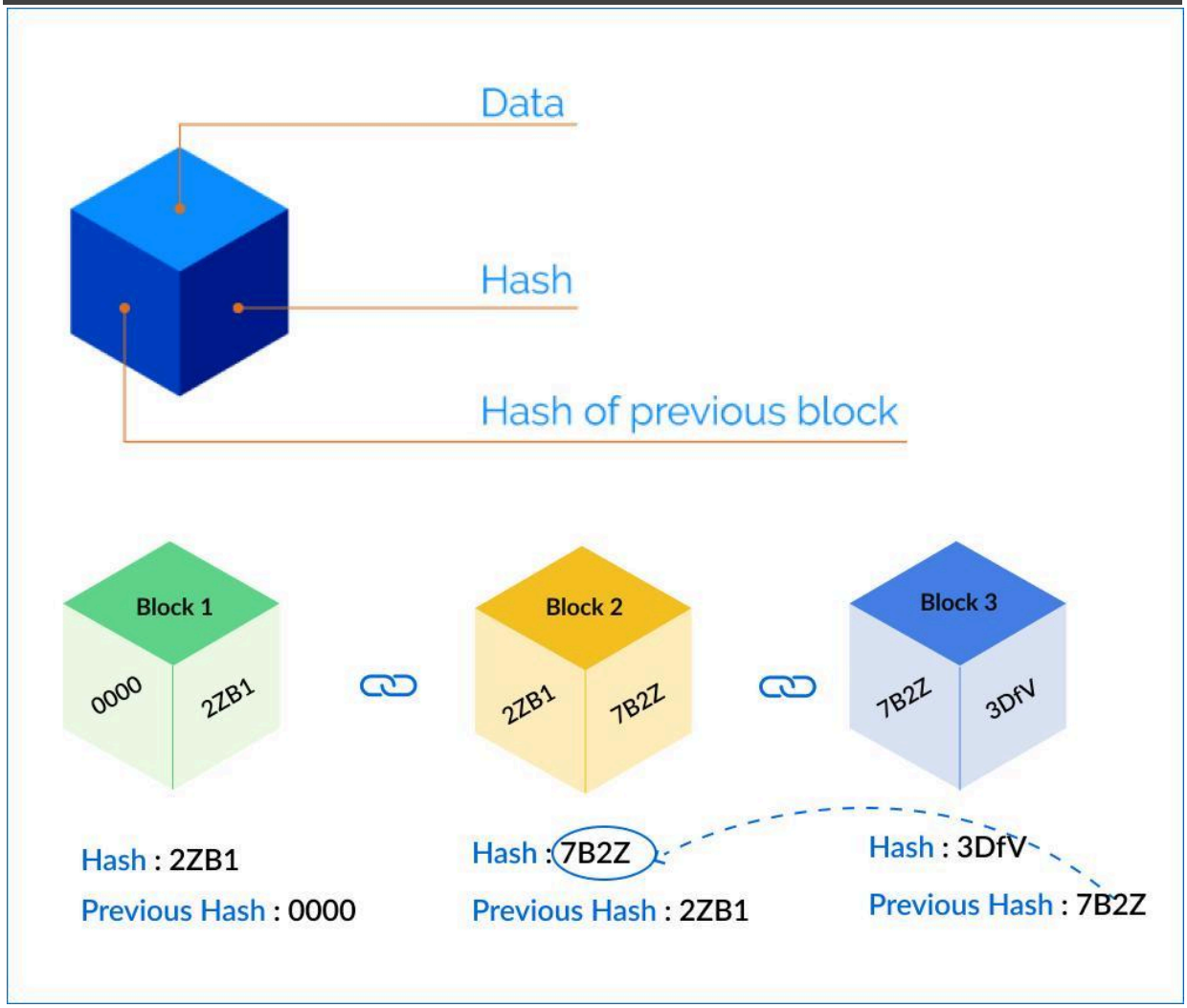
This combines finance, economics, and computer science. Users could test how different electricity rates, mining machines, and Bitcoin price assumptions affect operating profitability.

CoinWarz

offers an existing example to research.

CoinWarz

+1



3. Blockchain Security and Tampering Simulator

This is the most directly related to programming. Users could edit block data, and your app would demonstrate why the stored hashes no longer match.

MIT's interactive blockchain demonstration

provides an excellent prior implementation to research.

blockchain.mit.edu

+2

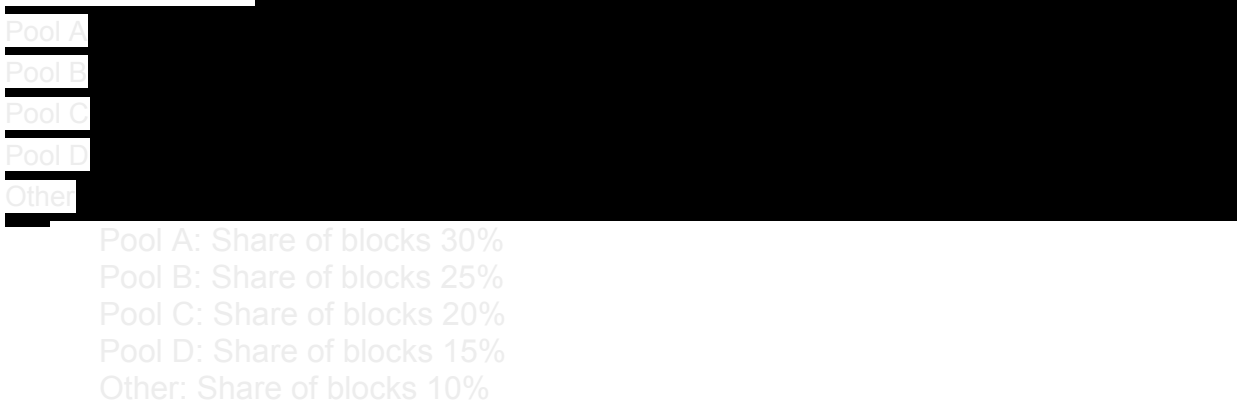
How my top choice would work in CISC106

Bitcoin Mining Pool Concentration Analyzer

Example web application structure

Sample mining pool data

Illustrative values, not live Bitcoin statistics



Hypothetical distribution for demonstration

Try changing the simulated mining distribution

Pool A

30%

Pool B

25%

Pool C

20%

Other pools

25%

Pools exceeding 50% combined

2 of 4

Based on the simulated shares above. Combined pool shares do not establish coordinated control or prove an attack is possible.

For your actual project, you could use a simple preloaded dataset rather than retrieving live information. Your Python code would process a list of mining pool records, calculate each pool's share, sort the results, and count how many of the largest pools exceed a combined 50%.

A `MiningPool` dataclass might contain the pool's name, number of blocks found, observation start date, and observation end date. You would not need to model actual mining equipment, transactions, or individual miners.

This demonstrates several CISC106 concepts: dataclasses, lists, loops, functions, conditionals, sorting, and calculations.

One important research distinction: mining pool concentration is a useful indicator, but it does not independently prove who controls the miners or the network.

My recommendation

I'd rank Bitcoin Mining Decentralization Analyzer first because it gives you a genuinely interesting research question and plenty of meaningful Python processing without requiring complex blockchain infrastructure.

If you want the easiest path to a working project, choose the Bitcoin Transaction Fee Calculator. If you want the most visually interesting programming demonstration, choose the Blockchain Security and Tampering Simulator.

For Part 5 of the worksheet, a useful question to explore next would be:

I'm interested in building a Bitcoin mining decentralization analyzer using Python and Drafter for an introductory computer science course. What three calculations would best help beginners understand mining pool concentration, and which advanced features should I avoid?

Your worksheet requires you to push back on an overly complicated suggestion, refine your project idea, and include the complete LLM conversation in Part 7. Be sure to write your worksheet responses in your own words.

CS1 Final Project 0 - Pick a Problem.pdf

Which direction interests you most: Bitcoin mining economics, decentralization

research, or blockchain security and consensus?

okay im going to pick project 1. bitcoin mining decentralization analyzer. what would the goals of the project be? what existing tools are already out there whats the input, process, and output?

Your Bitcoin Mining Decentralization Analyzer would be a web application that helps users understand how Bitcoin mining power is distributed among mining pools, and whether a small number of pools account for a large percentage of Bitcoin blocks.

I would frame your project around one central research question:

How concentrated is Bitcoin mining among the largest mining pools, and what can that tell us about Bitcoin's decentralization?

Here's how I'd structure it for your CISC106 worksheet:

Part 1: Project goals and purpose

1.1 What problem are you trying to solve?

Bitcoin is designed to function without a single central authority. However, many individual miners participate in mining pools, which coordinate mining activities.

That raises an important question: How decentralized is Bitcoin mining if a few large pools produce most of the blocks?

Your application would make it easier for students, researchers, and Bitcoin enthusiasts to investigate mining concentration using simple statistics and visualizations.

[redacted]

Main goals

1. Measure mining concentration: Calculate the percentage of observed Bitcoin blocks produced by each mining pool.
2. Identify major mining pools: Rank pools based on their contribution to total blocks mined.
3. Analyze decentralization: Calculate how much of the total mining activity is represented by the top 3 or 5 pools.
4. Explore concentration thresholds: Show how many of the largest pools collectively account for more than 50% of blocks in a dataset.
5. Educate users: Explain what these statistics suggest about mining decentralization, including their limitations.

A key limitation is that block production is an estimate of mining power, not a perfect measurement. Several pools exceeding 50% collectively does not mean they are acting together or controlling the network.

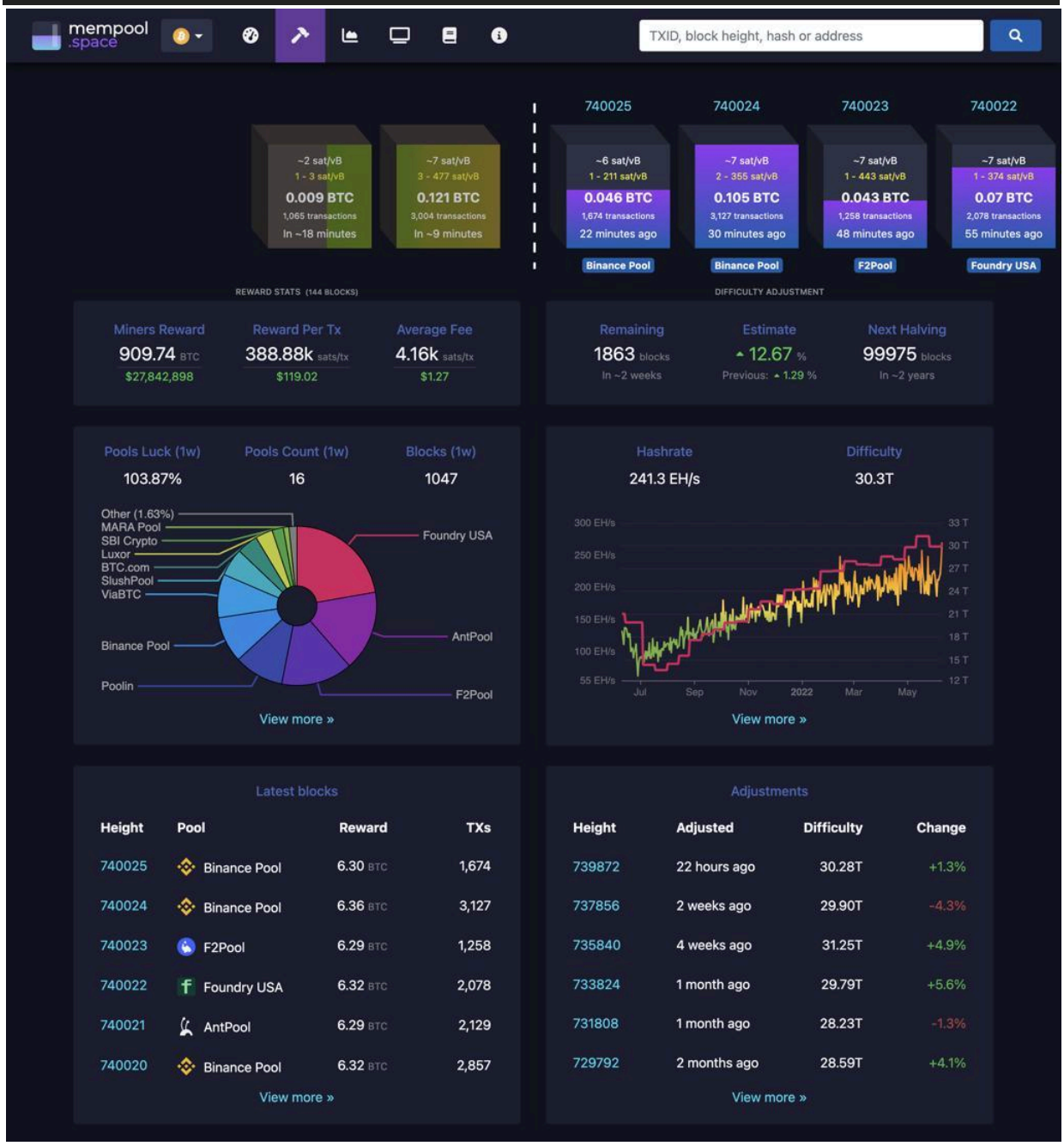
[redacted]

1.2 Existing tools and websites

[redacted]

I found three useful examples for your research section.

[redacted]



1. mempool.space

Mining dashboard

Displays Bitcoin mining pool rankings, blocks mined, pool shares, network hashrate estimates, and difficulty statistics. This would be an excellent source for the data

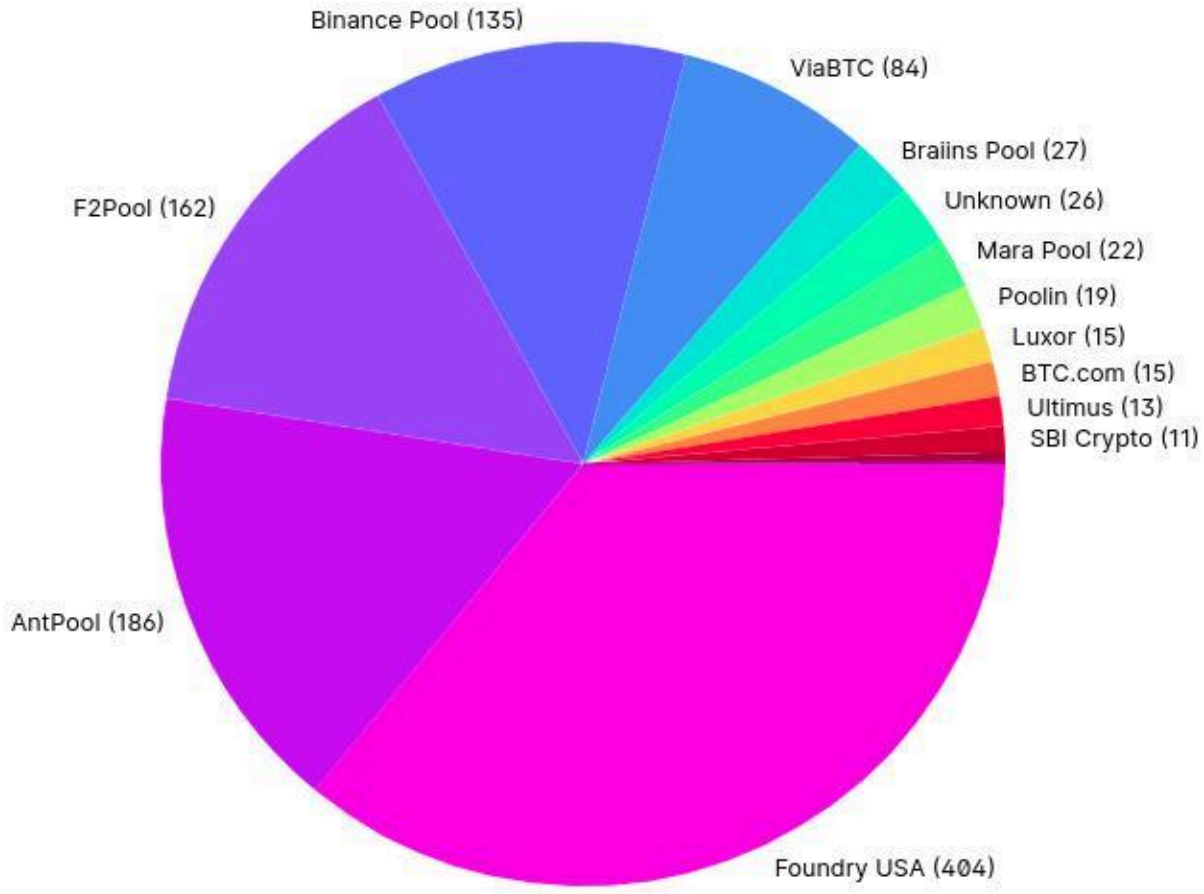
behind your project.

Mempool.space

Hashrate Distribution

An estimation of hashrate distribution amongst the largest mining pools.

24H 2D 4D **7D** 10D 6M 1Y 2Y 3Y

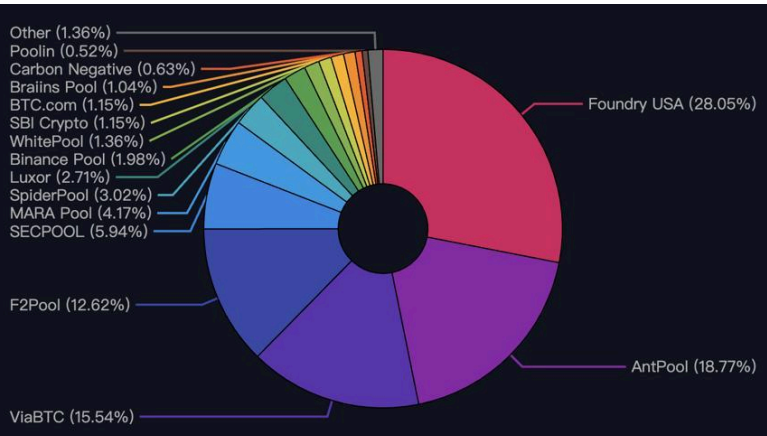


2. Blockchain.com Explorer

Hashrate distribution

Shows estimated mining pool distribution, historical data, and comparisons across time periods. It also explains limitations in identifying which pool mined a block.

Charts - Hashrate Distribution



Rank	Pool	Hashrate	Blocks	Avg Health	Avg Block Fees	Empty Blocks
1	Foundry USA	217.95 EH/s	269 (28.05%)	99.99%	-1.86%	0 (0.00%)
2	AntPool	145.84 EH/s	180 (18.77%)	99.99%	-2.98%	0 (0.00%)
3	ViaBTC	120.72 EH/s	149 (15.54%)	99.99%	-1.63%	0 (0.00%)
4	F2Pool	98.04 EH/s	121 (12.62%)	99.99%	-3.65%	0 (0.00%)
5	SECPOOL	46.18 EH/s	57 (5.94%)	99.99%	-4.00%	0 (0.00%)

3. Hashrate Index

Mining pool comparison

Provides mining pool rankings, estimated hashrate, market share, block counts, and historical comparisons. Its interface is intended for more detailed mining analysis.

HashrateIndex

What would make your project different? Rather than building another large mining

dashboard, your application would focus on teaching users how concentration is calculated and letting them explore different concentration thresholds.

Part 2: Input, Process, Output (IPO)

INPUT

User selects a mining dataset, chooses how many leading pools to analyze, and sets a concentration threshold.

PROCESS

Python loops through pool data, totals mined blocks, calculates percentages, sorts pools, and checks combined shares against the threshold.

OUTPUT

A mining pool ranking, each pool's share, combined concentration percentage, and a plain-language explanation.

2.1 Input: What does the user provide?

I would keep the inputs limited to three things.

Input	Example
Dataset or observation period	A preloaded weekly mining dataset
Number of largest pools to analyze	Top 3 pools
Concentration threshold	50%

For CISC106, I recommend using preloaded mining pool data, rather than building a live API connection. You could collect a snapshot from mempool.space and save the pool names and block counts in a Python list or CSV file.

2.2 Processing: What will the Python program do?

Your Python application would perform these operations:

1. Read the mining pool data into a list of dataclasses.
2. Calculate the total number of blocks mined in the dataset.
3. Calculate each pool's percentage of total blocks.
4. Sort the mining pools from largest to smallest.
5. Add together the percentages of the top selected pools.
6. Compare the combined percentage with the user's chosen threshold.
7. Produce a result and interpretation using conditional statements.

The most important formula would be:

$$\text{Pool Share} = \frac{\text{Blocks mined by pool}}{\text{Total blocks mined}} \times 100$$

For example, if Pool A mined 250 of 1,000 blocks:

$$\frac{250}{1000} \times 100 = 25\%$$

1000

250

×100=25%

2.3 Output: What will the user see?

Here is an example based on observed one-week pool shares from mempool.space at the time of this research.

Mempool.space

Mining Concentration Results

Example using observed one-week data

0%7%14%21%28%

Foundry USA

AntPool

F2Pool

Foundry USA: Observed block share 26.5%
AntPool: Observed block share 19.6%
F2Pool: Observed block share 16.9%

Top 3 combined share

63.01%

Compared with threshold

Above 50%

Interpretation: These three pools collectively produced approximately 63% of observed blocks

during this period. This indicates concentrated pool-level activity but does not establish shared ownership or coordinated control.

Part 3: Data abstraction

Although you didn't ask about this part yet, it's important because your rubric awards three points for defining the information your program needs to store.

I would use a `MiningPool` dataclass.

Attribute	Data type	Purpose
`name`	`str`	Name of mining pool
`blocks_mined`	`int`	Number of blocks produced
`period`	`str`	Observation period
`share`	`float`	Calculated percentage of blocks

You could ignore individual miners, mining machine models, electricity consumption, and transaction details because none are necessary to calculate block-production concentration.

What I would actually build

I would make your first version contain just four features: a selectable preloaded dataset, a ranking table, a combined top-pool concentration calculation, and an explanation of the results.

Once those work, you could add simple bar charts and support multiple historical periods. I would avoid automatic blockchain data collection, real-time updates, and

attempts to predict 51% attacks for your first implementation.

This would be a legitimate, research-focused project while still demonstrating the CISC106 material you have learned: lists, loops, dataclasses, functions, conditionals, and arithmetic.

One research question worth considering as you continue: Is the concentration of mining pools a reliable measure of Bitcoin decentralization, given that individual miners can choose which pool to join?

That question would give your application a stronger academic purpose beyond simply calculating percentages.

Also, your assignment requires the complete AI conversation in Part 7, so make sure you preserve this discussion for your worksheet.

what are the difficulties that make this code complex, whats my coding strategy, and whast the ultimate goal for this project?

Here's how I would answer Part 4: Challenges, AI Strategy, & Goals for your Bitcoin Mining Decentralization Analyzer. I've written these in a natural, first-person style that you can adapt for your worksheet.

4.1 Difficulties: What parts of this project seem the most difficult or technically complex?

The most difficult part of this project would probably be organizing Bitcoin mining pool data and making sure my calculations are accurate. I would need to calculate each pool's percentage of mined blocks, sort the pools from largest to smallest, and

determine how much of the total mining activity is concentrated among the largest pools. Another challenge would be connecting these calculations to a functional web application using Drafter, especially making sure the results update correctly when users change their inputs. I would also need to make sure the application doesn't confuse mining pool concentration with actual control over the Bitcoin network.

4.2 Coding Strategy: How will you understand the code and avoid relying too heavily on AI?

I plan to break the project into smaller functions and build each part separately before connecting everything to Drafter. I'll start with a small dataset of mining pools and write functions to calculate percentages, sort the data, and compare mining concentration against a selected threshold. I'll use Bakery tests to verify my calculations with known answers and test unusual inputs, such as empty datasets or zero blocks. If I use AI, I'll mainly use it to explain concepts, review my code, and help identify errors rather than simply copying generated solutions. I'll also review each function myself and make sure I understand how it works before adding it to the application.

4.3 Goals: What is your ultimate goal for this project?

My ultimate goal is to create an educational web application that helps users better understand Bitcoin mining decentralization through data analysis. I want users to be able to compare mining pools, see how mining activity is distributed, and understand whether a small number of pools account for a large percentage of Bitcoin blocks. Personally, I also want to improve my Python programming skills, become more comfortable with Drafter, and apply what I've learned in CISC106 to a subject I'm genuinely interested in: blockchain technology and Bitcoin.

Why these answers fit the rubric

- Difficulties: Identifies specific programming challenges involving data, calculations, sorting, and Drafter.
- Coding strategy: Explains your development process, how you'll test functions using Bakery, and how you'll use AI responsibly.
- Ultimate goal: Connects the educational purpose of the website with your personal interest in Bitcoin and your programming development.

One thing I particularly recommend keeping is the distinction between mining pool [redacted] concentration and control over the Bitcoin network. It shows that you understand the research question and the limitations of what your application can actually measure. [redacted]

These answers address all three questions in Part 4. You can revise the wording to reflect your own approach before submitting. [redacted]

in the project, what will the site's diagram look like? what functions will be used. I was thinking i will use a home() for the welcome, configure() to input the fields and reset, results() for the mning pool ranking nd analysis and through that it will either be an option for pool_details() the mining pool's info, and the about() for the methods and limits. with these value types. pools: list[MiningPool]

top_count: int

threshold: float

selected_pool: str

combined_share: float

can you also provide to me the ate dataclassattributes (min 4), the second data class, routes (min 7), pages (5), and user input (min 3).

Worked for 54s

Your proposed structure is a good fit for the Bitcoin Mining Decentralization Analyzer. I would keep your five pages and add two action routes: one that processes the analysis and one that resets the settings.

That gives you 7 routes, 5 pages, at least 3 user inputs, and 2 problem-related dataclasses, plus the `State` dataclass that Drafter uses.

1. Website diagram



Proposed navigation structure. The action routes process or reset data and return one of the five visible pages.

One important Drafter detail: The front page is normally named `index()`, not `home()`. It can still display "Home" to the user. Drafter uses `@route` functions to generate pages and navigate between them.

Drafter Documentation

+1

2. Routes (minimum 7)

Route function	Purpose	Returns
`index()`	Welcomes the user and introduces mining decentralization	Home
`configure()`	Displays inputs for period, top-pool count, and threshold	Configure

`run_analysis()`	Validates inputs and performs the calculations	Results
`results()`	Displays mining pool rankings, percentages, and findings	Results
`pool_details()`	Displays information about a selected mining pool	Pool Details
`about()`	Explains calculations, data sources, and limitations	About
`reset()`	Restores the default settings and clears previous selections	Configure

The `run_analysis()` route can update the application's state and then call `results(state)`. This is consistent with the route pattern in Drafter's documentation.



Drafter Documentation

3. Pages (5 total)

Page 1: Home

Displays the application name, a short explanation of Bitcoin mining pools, and buttons for "Start Analysis" and "About."

Page 2: Configure

Lets users select their dataset, the number of largest pools to examine, and the concentration threshold. Includes "Run Analysis" and "Reset" controls.

Page 3: Results

Displays the ranked mining pools, their percentages, combined concentration, and whether the selected threshold was exceeded. Users can choose a pool to learn more.

Page 4: Pool Details

Displays the selected pool's name, blocks mined, percentage of the total, and a brief description of the pool.

Page 5: About

Explains the calculation methods, data sources, and limitations, especially why mining pool share does not necessarily equal independent control over Bitcoin's hashpower.

4. Dataclasses and attributes

I suggest two data-related dataclasses and one `State` dataclass. This makes the design especially clear if your professor expects two dataclasses in addition to the Drafter state.

Dataclass 1: `MiningPool`

Represents one mining pool.

Attribute	Type	Description
`name`	`str`	Mining pool name
`blocks_mined`	`int`	Blocks found during the observation period
`share`	`float`	Calculated percentage of observed blocks
`description`	`str`	Brief background information for the details page

Dataclass 2: `MiningDataset`

Represents one collection of mining pool observations.

Attribute	Type	Description
`name`	`str`	Dataset name
`period`	`str`	Dates or time window measured
`source`	`str`	Where the original data came from
`pools`	`list[MiningPool]`	Mining pools included in that dataset

For example, you could have separate saved datasets representing two historical observation periods, allowing users to compare them without connecting to a live API.

Drafter `State` dataclass

Your original five state values are appropriate. I would add `period` so the application remembers which dataset is selected.

```
@dataclass
class State:
    pools: list[MiningPool]
    top_count: int
    threshold: float
    selected_pool: str
    combined_share: float
    period: str
```

Use percentage points consistently. For instance, `threshold = 50.0` represents 50%, and `combined_share = 62.5` represents 62.5%, not 0.625.

The `State` dataclass is important because Drafter uses it to remember information as the user moves between routes.



Drafter Documentation

5. User inputs (minimum 3)

You could use these four inputs, with the first three on the Configure page.

Example configuration form

1. Observation period

2. Number of top mining pools

3. Concentration threshold (%)

4. Selected mining pool (Results page)

This is an interactive planning example, not your completed Drafter application. Dataset labels and pool choices are illustrative.

In Drafter, these inputs could be `SelectBox` and `TextBox` components. Their names should match the parameters of the route receiving the values, such as `top_count: int` and `threshold: float`.



Drafter Documentation

6. Helper functions

Besides your seven route functions, I would plan these Python functions for the actual calculations.

Function	What it does
`load_dataset(period)`	Finds the saved mining data for the selected period
`total_blocks(pools)`	Calculates the total number of blocks
`calculate_shares(pools)`	Calculates each mining pool's share of the total
`rank_pools(pools)`	Sorts mining pools by blocks mined, highest first
`calculate_combined_share(pools, top_count)`	Adds the shares of the largest selected pools
`check_threshold(combined_share, threshold)`	Determines whether the combined share exceeds the selected threshold
`find_pool(pools, selected_pool)`	Finds a specific mining pool for the details page

The distinction is that routes control navigation and pages, while helper functions handle the underlying calculations. Separating them makes your code easier to test with Bakery and reduces repetition.

7. Requirements checklist

At least 7 routes

7 planned

At least 5 pages

5 planned



Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.